

Estudio de la complejidad temporal de algoritmos recursivos de búsqueda, ordenamiento y selección

Franco Aguilar[†], Milton Hernandez[†] y Luis Muñoz[†]

[†]Universidad de Magallanes

Este informe fue compilado el 15 de mayo de 2026

Resumen

En este informe se presenta un estudio comparativo de la complejidad temporal de algoritmos de ordenamiento, búsqueda y selección basados en la estrategia de Divide y Vencerás, aplicados a la base de datos ficticia de deportistas desarrollada en el Proyecto 1.

Se llevaron a cabo análisis teóricos y experimentos sobre conjuntos de datos de hasta 50 000 elementos bajo tres escenarios de distribución distintos (elementos previamente ordenados, en orden inverso y en orden aleatorio). Para los algoritmos de ordenamiento se implementó Merge Sort en su versión clásica y optimizada con umbral de Insertion Sort, junto con Quick Sort bajo cuatro estrategias de selección de pivote basadas en el esquema de partición de Lomuto. Los algoritmos de búsqueda incluyen búsqueda binaria recursiva, búsqueda binaria para rangos, búsqueda exponencial y búsqueda por interpolación. Finalmente, se implementó Quick Select como algoritmo de selección del k -ésimo elemento.

Los resultados corroboraron la superioridad de los algoritmos $O(n \log n)$ frente a los algoritmos $O(n^2)$ de la tarea anterior, y evidenciaron el impacto que tienen las optimizaciones (umbral en Merge Sort, elección de pivote en Quick Sort) sobre el rendimiento práctico dentro de una misma clase de complejidad.

Keywords: Divide y Vencerás, Complejidad temporal, Merge Sort, Quick Sort, Búsqueda binaria, Quick Select.

Índice

1	Introducción	3
2	Objetivos	3
3	Desarrollo	4
3.1	Quicksort	4
	Proceso de partición • Selección de QuickSort para experimentación • Análisis de complejidad mediante el Teorema Maestro	
3.2	Quick Select: búsqueda del k -ésimo elemento	7
	Mejor caso y caso promedio • Peor caso • Comentario general	
3.3	Merge Sort: versión clásica y versión optimizada	9
	Función Merge: la operación clave • Merge Sort clásico • Merge Sort optimizado con <i>threshold</i> • Análisis experimental del umbral • Comentario general	
3.4	Búsqueda Binaria Recursiva	13
	Análisis de complejidad • Comentario general	

3.5	Búsqueda Binaria por Rango	14
	Búsqueda por valor exacto: <code>first</code> y <code>last</code> • Búsqueda por intervalo: <code>lower_bound</code> y <code>upper_bound</code> • Idea general en pseudocódigo • Análisis de complejidad • Comentario general	
3.6	Búsqueda Exponencial	16
	Funcionamiento general • Análisis de complejidad • Comentario general	
3.7	Búsqueda por Interpolación	18
	Funcionamiento general • Análisis de complejidad • Comentario general	
4	Comparación de algoritmos de ordenamiento	20
5	Comparación de algoritmos de búsqueda	21
6	Aplicación práctica	24
6.1	Interfaz de línea de comandos	24
6.2	Justificación de la elección de algoritmos	26
	Algoritmo de ordenamiento: Quick Sort • Algoritmo de búsqueda: búsqueda por interpolación y búsqueda por rangos	
7	Conclusiones	27
8	Contribución por integrante	29
9	Anexos	32
9.1	Experimentos con Quicksort	33
9.2	Teorema Maestro	34

1. Introducción

El diseño de algoritmos tiene como objetivo encontrar soluciones a problemas de manera eficiente, considerando las limitaciones de tiempo y memoria presentes en los sistemas informáticos. En el proyecto anterior se estudiaron algoritmos iterativos de ordenamiento y búsqueda verificando experimentalmente su comportamiento cuadrático o lineal. Si bien esos resultados confirmaron la correspondencia entre el análisis teórico y el experimental, los algoritmos $O(n^2)$ presentan limitaciones evidentes frente a conjuntos de datos de gran tamaño.

El presente informe aborda el mismo problema de ordenamiento y búsqueda sobre la base de datos ficticia de deportistas, esta vez desde la perspectiva de la estrategia *Divide y Vencerás*: un paradigma algorítmico que descompone el problema en subproblemas más pequeños del mismo tipo, los resuelve de forma recursiva y combina sus soluciones para obtener el resultado final. Esta estrategia da origen a algoritmos como Merge Sort y Quick Sort, que alcanzan una complejidad de $O(n \log n)$ en el caso promedio, y a variantes de búsqueda como la búsqueda exponencial y por interpolación, que extienden la búsqueda binaria para escenarios específicos.

Adicionalmente, se analiza Quick Select como algoritmo de selección del k -ésimo elemento, que permite resolver este problema en tiempo promedio $O(n)$ sin necesidad de ordenar el arreglo completo.

2. Objetivos

Analizar y comparar el comportamiento de algoritmos de ordenamiento, búsqueda y selección basados en la estrategia de Divide y Vencerás, estableciendo una correspondencia entre los resultados teóricos y experimentales, y evaluando su rendimiento frente a los algoritmos iterativos estudiados en el Proyecto 1.

1. Comprender el funcionamiento y la implementación de los algoritmos de Divide y Vencerás utilizados, incluyendo sus variantes y optimizaciones.
2. Verificar experimentalmente las complejidades teóricas, evaluando el impacto de distintos parámetros sobre el rendimiento práctico de los algoritmos.
3. Integrar los algoritmos implementados al sistema de deportistas del Proyecto 1, incorporando funcionalidades de ordenamiento, búsqueda por rango y selección del k -ésimo elemento.

3. Desarrollo

3.1. Quicksort

Este algoritmo, al igual que Merge Sort, consta de dos elementos esenciales: la **partición** y el **ordenamiento** como tal. Partiendo de un arreglo desordenado, la manera de ordenarlo con este algoritmo es la siguiente:

1. Se escoge un *pivote* dentro del arreglo.
2. Se ordena el arreglo alrededor de ese pivote, generando “dos nuevos arreglos” (que en realidad son dos partes del original): a la izquierda del pivote quedan los elementos menores que él (no necesariamente ordenados) y a la derecha los mayores que él (tampoco ordenados).
3. Se repiten los dos primeros pasos sobre cada uno de estos subarreglos.
4. Así sucesivamente, de manera recursiva, hasta tener el arreglo completamente ordenado.

A nivel de pseudocódigo, el ordenamiento sigue la siguiente estructura:

Algorithm 1 Quicksort $\triangleright V[0..n - 1]$

```

1: procedure QUICKSORT( $V, left, right$ )
2:   if ( $right - left + 1 < 2$ ) then
3:     return // Arreglos de 1 o 0 elementos ya están ordenados
4:    $p \leftarrow \text{Pivote}(V, left, right)$  // Por el método escogido
5:    $p \leftarrow \text{LomutoPartition}(V, left, right, p)$ 
6:   Quicksort( $V, left, p - 1$ )
7:   Quicksort( $V, p + 1, right$ )

```

Selección del pivote

La elección del pivote impacta directamente en el rendimiento del algoritmo. Para este proyecto se implementaron las cuatro estrategias clásicas para esta selección:

1. **Primer elemento del subarreglo:** se toma como pivote el elemento ubicado en la posición `left`. Es la opción más simple, pero degenera fácilmente cuando el arreglo se encuentra previamente ordenado o en orden inverso.
2. **Último elemento del subarreglo:** se toma el elemento en la posición `right`. Presenta el mismo problema que la opción anterior frente a entradas patológicas.
3. **Elemento aleatorio:** se selecciona una posición al azar dentro del rango $[left, right]$. Esta estrategia rompe la dependencia respecto al orden inicial del arreglo, ofreciendo un rendimiento esperado de $O(n \log n)$ con alta probabilidad.
4. **Mediana de tres:** se observan los valores en las posiciones `left`, `right` y la mitad del subarreglo, y se toma como pivote el que se encuentra en la mediana de los tres. Esta estrategia tiende a evitar pivotes extremos que produzcan particiones muy desbalanceadas.

3.1.1. Proceso de partición

Existen distintos esquemas de partición para Quicksort, siendo los más conocidos los de *Hoare* y *Lomuto*. En el presente informe se utilizó el esquema de partición de Lomuto, por ser el solicitado en el enunciado del proyecto y por su simplicidad de implementación.

El procedimiento ubica primero el pivote al final del subarreglo (mediante un intercambio inicial) y, a continuación, recorre el rango $[left, right - 1]$ moviendo al inicio todos los elementos menores o iguales al pivote. Al finalizar el recorrido, el pivote se reubica entre los elementos menores y los mayores, retornando su posición final.

Algorithm 2 Partición de Lomuto $\triangleright V[0..n - 1]$

```

1: procedure LOMUTOPARTITION( $V, left, right, pivot$ )
2:   swap( $V[pivot], V[right]$ )
3:    $l \leftarrow left - 1$ 
4:   for  $i \leftarrow left$  to  $right - 1$  do
5:     if  $V[i] \leq V[right]$  then
6:        $l \leftarrow l + 1$ 
7:       swap( $V[l], V[i]$ )
8:   swap( $V[l + 1], V[right]$ ) // Pivote en su posición final
9:
10:  return  $l + 1$ 

```

3.1.2. Selección de QuickSort para experimentación

Para comparar Quicksort con los demás algoritmos del proyecto resultaba inviable mantener las cuatro variantes de pivote en igualdad de condiciones, por lo que se decidió escoger una única variante representativa. Para tomar esta decisión de manera fundamentada, se diseñó el siguiente procedimiento experimental¹:

1. **Arreglo desordenado**: se ejecutaron las cuatro variantes sobre un arreglo aleatorio de 1 000 000 de elementos, evaluado en 25 puntos equiespaciados, con 1000 repeticiones por punto.
2. **Arreglo previamente ordenado**: se ejecutaron las cuatro variantes sobre un arreglo de 50 000 elementos en orden ascendente, evaluado en 25 puntos equiespaciados, con 500 repeticiones por punto. Este escenario se incluyó porque corresponde al peor caso documentado de Quicksort cuando se utilizan los pivotes *primer elemento* o *último elemento*.
3. **Arreglo en orden inverso**: con los mismos parámetros del experimento anterior, este escenario también constituye un peor caso para los pivotes *primer elemento* y *último elemento*.

La razón por la que un arreglo ya ordenado fuerza un comportamiento $O(n^2)$ al usar el último elemento como pivote es que, en cada llamada recursiva, el pivote elegido resulta ser el mayor del subarreglo. La partición devuelve entonces un subarreglo izquierdo de tamaño $n - 1$ y uno derecho vacío: el arreglo se recorre completo pero su estado no cambia, y este patrón se repite en todas las pasadas. La profundidad de la recursión escala linealmente con n y el trabajo total se vuelve proporcional a n^2 . Un razonamiento análogo aplica al caso del arreglo en orden inverso al usar el primer elemento como pivote.

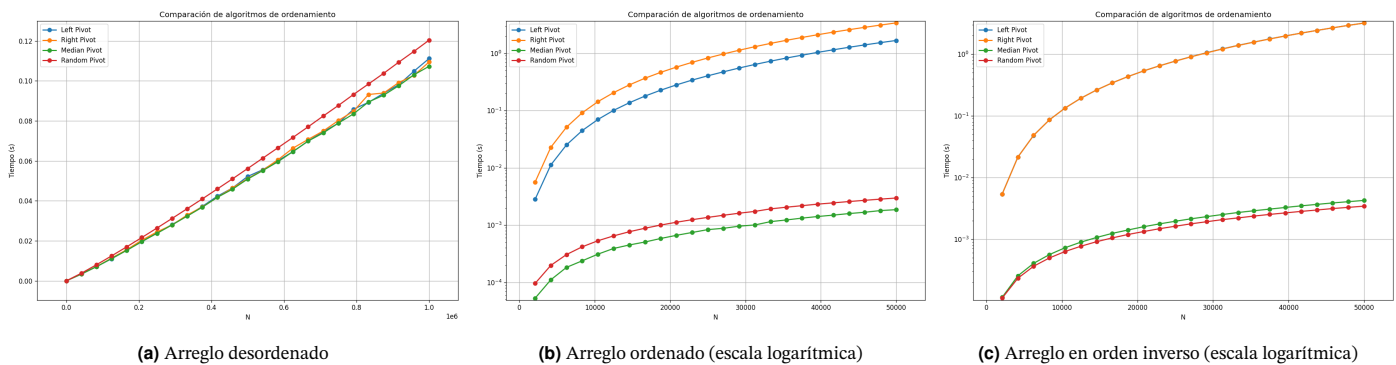


Figura 1. Comparación de las cuatro variantes de pivote de Quicksort sobre los tres escenarios experimentales utilizados para la selección.

Los resultados completos de cada uno de estos experimentos se presentan en las tablas 1, 2 y 3, donde se reportan las tablas detalladas con los tiempos promedio por variante y por tamaño de entrada. A partir de estos datos se desprenden las siguientes observaciones:

¹Si desea experimentar con las distintas particiones puede ir a [este commit](#) donde el experimento entrega resultados equivalentes a los usados en este proceso. Además el código fuente 1 muestra la implementación de la función de mediana de tres y la elección del pivote.

- En los casos de arreglo previamente ordenado y arreglo en orden inverso, los pivotes **primer elemento** y **último elemento** se degradan claramente a un comportamiento $O(n^2)$, mientras que las variantes de **mediana de tres** y **aleatorio** mantienen un comportamiento similar entre sí, conservando un orden de crecimiento del tipo $O(n \log n)$.
- En el escenario de arreglo desordenado, la variante de **pivote aleatorio** muestra un rendimiento peor que el de las otras tres variantes.

A partir de estas observaciones, se decidió utilizar la variante de **mediana de tres** como pivote por defecto en la integración final del programa, dado que mantiene un buen comportamiento tanto en escenarios desordenados como en escenarios patológicos para los pivotes deterministas básicos².

3.1.3. Análisis de complejidad mediante el Teorema Maestro

La complejidad temporal del Quick Sort puede determinarse formalmente a través del Teorema Maestro (véase Anexo 9.2). En su caso promedio, el esquema de partición de Lomuto divide el arreglo en dos subarreglos de tamaño aproximadamente $n/2$, generando así **dos subproblemas** ($a = 2$) cada uno de tamaño $n/2$ ($b = 2$). El costo adicional corresponde al recorrido completo del arreglo para realizar la partición, operación que tiene un costo de orden $O(n)$; es decir, $f(n) = cn^1$, por lo que $k = 1$. Identificados los parámetros, la recurrencia del caso promedio queda expresada como:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

Aplicando la versión simplificada del Teorema Maestro se calcula $\log_b a = \log_2 2 = 1 = k$. Al cumplirse $\log_b a = k$, se está ante el segundo caso del teorema, por lo que:

$$T(n) \in O(n^k \log n) = O(n \log n)$$

Este resultado coincide con el comportamiento observado experimentalmente y confirma que, bajo una elección de pivote que produce particiones equilibradas, Quick Sort exhibe una complejidad temporal de $O(n \log n)$ en el caso promedio. Cabe destacar que el peor caso (provocado por pivotes que generan particiones completamente desbalanceadas, como ocurre con el pivote del primer o último elemento sobre arreglos ya ordenados) produce la recurrencia $T(n) = T(n - 1) + cn$, cuya solución es $O(n^2)$, comportamiento que no puede caracterizarse directamente mediante el Teorema Maestro al no satisfacer la forma $T(n/b)$.

²Si bien la mediana de tres es la opción más estable de las evaluadas, existe un patrón de entrada adversarial conocido como *median-of-three killer*, descrito por McIlroy [1], capaz de forzar a esta variante a un comportamiento $O(n^2)$. Se trata de un caso muy específico que excede el alcance del análisis experimental presentado.

3.2. Quick Select: búsqueda del k -ésimo elemento

Quick Select es un algoritmo que permite encontrar un elemento según la posición que tendría si el arreglo estuviera ordenado. Por ejemplo, puede encontrar el k -ésimo menor o el k -ésimo mayor elemento sin ordenar todos los datos.

La idea principal es que, si solo se necesita un elemento, no hace falta ordenar el arreglo completo. El algoritmo va descartando las partes donde ese elemento no puede estar, hasta quedarse con la parte correcta.

Para entenderlo, se pueden considerar estas ideas:

1. Si el arreglo estuviera ordenado, el k -ésimo menor estaría en la posición $V[k - 1]$.
2. Si se busca el k -ésimo mayor, estaría en la posición $V[n - k]$.
3. Después de particionar, el pivote queda en la posición que tendría dentro del arreglo ordenado.

El funcionamiento es parecido al de Quick Sort (Sección 3.1). Primero se escoge un pivote y se particiona el arreglo: los elementos menores quedan a un lado y los mayores al otro. Luego se revisa la posición final del pivote. Si esa posición es la que se busca, el algoritmo termina. Si la posición buscada está a la izquierda, se continúa solo por la izquierda. Si está a la derecha, se continúa solo por la derecha.

La diferencia con Quick Sort es importante: Quick Sort debe ordenar ambas partes, mientras que Quick Select solo sigue trabajando en una. Por eso evita trabajo innecesario cuando solo se busca un elemento.

A nivel de pseudocódigo, el algoritmo se resume de la siguiente forma:

Algorithm 3 Quick Select $\triangleright V[0..n - 1]$

```

1: procedure QUICKSELECT( $V, left, right, k$ )
2:   if  $left = right$  then
3:     return  $V[left]$ 
4:    $pivot \leftarrow \text{MedianOfThree}(V, left, right)$  // Elección del pivote
5:    $pivot \leftarrow \text{LomutoPartition}(V, left, right, pivot)$ 
6:   if  $k = pivot$  then
7:     return  $V[pivot]$  // El pivote era el  $k$ -ésimo elemento
8:   else if  $k < pivot$  then
9:     return QuickSelect( $V, left, pivot - 1, k$ ) // Buscar a la izquierda
10:  else
11:    return QuickSelect( $V, pivot + 1, right, k$ ) // Buscar a la derecha
12:
13:
14:
15:
```

3.2.1. Mejor caso y caso promedio

El mejor caso ocurre cuando el pivote queda cerca del centro del arreglo. En ese caso, después de cada partición se descarta una parte grande de los datos y el problema se reduce rápidamente.

El trabajo total puede representarse así:

$$T(n) = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots$$

Esto significa que primero se revisan n elementos, luego aproximadamente la mitad, después un cuarto, y así sucesivamente. Al sumar esos costos, el crecimiento total es lineal. Por eso, en el mejor caso y en el caso promedio:

$$T(n) \in O(n)$$

En palabras simples, Quick Select puede ser muy rápido porque no ordena todo el arreglo, sino que elimina partes que ya no sirven para encontrar el elemento buscado.

3.2.2. Peor caso

El peor caso ocurre cuando el pivote queda siempre en una posición desfavorable, por ejemplo, como el menor o el mayor elemento. En esa situación casi no se descartan datos, por lo que el algoritmo debe seguir revisando casi todo el arreglo en cada paso.

El costo se puede expresar como:

$$T(n) = n + (n - 1) + (n - 2) + \cdots + 1$$

Esta suma crece de forma cuadrática, por lo que el peor caso es:

$$T(n) \in O(n^2)$$

Aunque este peor caso existe, en la práctica Quick Select suele funcionar bien, especialmente si se usa una estrategia de pivote más cuidadosa, como la mediana de tres.

3.2.3. Comentario general

En resumen, Quick Select busca un elemento por posición sin ordenar completamente el arreglo. Usa la misma idea de partición de Quick Sort, pero solo continúa por el lado donde puede estar la respuesta. Por eso, cuando los pivotes son razonables, su tiempo esperado es $O(n)$.

Sobre la verificación experimental

No se realizó un experimento separado para Quick Select porque su comportamiento depende principalmente de la elección del pivote, igual que en Quick Sort. Como esa parte ya fue analizada en la Sección 3.1.2, sus conclusiones también ayudan a entender el comportamiento de Quick Select.

3.3. Merge Sort: versión clásica y versión optimizada

Merge Sort es un algoritmo de ordenamiento basado en Divide y Vencerás. Su idea principal es dividir el arreglo en partes más pequeñas, ordenar esas partes y luego unir las nuevamente de forma ordenada. En vez de ordenar todo directamente, primero separa el problema y después reconstruye la solución.

En este proyecto se trabajó con dos versiones de Merge Sort. La primera es la versión clásica, que divide el arreglo hasta llegar a partes muy pequeñas y luego las mezcla. La segunda es una versión optimizada, que usa Insertion Sort cuando los subarreglos son pequeños para evitar llamadas recursivas innecesarias.

3.3.1. Función Merge: la operación clave

La función *merge* es la parte más importante de Merge Sort. Su trabajo es unir dos subarreglos que ya están ordenados y formar con ellos un solo subarreglo también ordenado.

La idea es sencilla: se compara el primer elemento disponible de cada subarreglo y se copia el menor al arreglo temporal. Luego se avanza en el subarreglo del cual se tomó ese elemento. Este proceso se repite hasta que uno de los dos subarreglos se termina. Finalmente, si quedan elementos en alguno de los lados, se copian directamente, porque ya estaban ordenados.

Algorithm 4 Merge $\triangleright V[left..right]$

```

1: procedure MERGE( $V, left, mid, right, comp\_f$ )
2:    $i \leftarrow left$ 
3:    $j \leftarrow mid + 1$ 
4:    $k \leftarrow 0$ 
5:   Crear arreglo temporal  $temp$  de tamaño  $(right - left + 1)$ 
6:   while  $i \leq mid$  and  $j \leq right$  do
7:     if  $comp\_f(V[i], V[j]) \leq 0$  then
8:        $temp[k] \leftarrow V[i]$ 
9:        $i \leftarrow i + 1$ 
10:    else
11:       $temp[k] \leftarrow V[j]$ 
12:       $j \leftarrow j + 1$ 
13:     $k \leftarrow k + 1$ 
14:   while  $i \leq mid$  do
15:      $temp[k] \leftarrow V[i]$ 
16:      $i \leftarrow i + 1$ 
17:      $k \leftarrow k + 1$ 
18:   while  $j \leq right$  do
19:      $temp[k] \leftarrow V[j]$ 
20:      $j \leftarrow j + 1$ 
21:      $k \leftarrow k + 1$ 
22:    $k \leftarrow 0$ 
23:   for  $i \leftarrow left$  to  $right$  do
24:      $V[i] \leftarrow temp[k]$ 
25:      $k \leftarrow k + 1$ 

```

La función *merge* tiene costo $O(n)$, porque cada elemento se revisa y se copia una cantidad constante de veces. Si entre ambos subarreglos hay n elementos, el trabajo total crece de forma lineal. Además, necesita memoria auxiliar para guardar temporalmente los elementos mientras se mezclan.

3.3.2. Merge Sort clásico

Merge Sort clásico sigue siempre el mismo procedimiento. Primero divide el arreglo en dos mitades. Después ordena cada mitad usando nuevamente Merge Sort. Finalmente, mezcla ambas mitades con la función *merge*.

El proceso se detiene cuando el subarreglo tiene tamaño 0 o 1, porque en ese caso ya está ordenado. A partir de ahí, las partes pequeñas se van mezclando hasta reconstruir el arreglo completo en orden.

Algorithm 5 Merge Sort clásico ▷ $V[left..right]$

```

1: procedure MERGESORTCLASSIC( $V, left, right, comp\_f$ )
2:   if  $left \geq right$  then
3:     return
4:    $mid \leftarrow left + (right - left)/2$ 
5:   MergeSortClassic( $V, left, mid, comp\_f$ )
6:   MergeSortClassic( $V, mid + 1, right, comp\_f$ )
7:   Merge( $V, left, mid, right, comp\_f$ )
8: 
```

▷ Caso base: arreglo de tamaño 0 o 1

▷ Ordenar mitad izquierda

▷ Ordenar mitad derecha

▷ Mezclar ambas mitades

Su complejidad se puede explicar así: en cada nivel de recursión se procesan en total n elementos, porque las mezclas recorren todos los datos de ese nivel. Además, como el arreglo se divide siempre a la mitad, se generan aproximadamente $\log n$ niveles.

Por eso, el costo total es:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

Aplicando el Teorema Maestro (véase Anexo 9.2), se obtiene:

$$T(n) \in O(n \log n)$$

En palabras simples, Merge Sort trabaja $O(n)$ por nivel y tiene alrededor de $\log n$ niveles. Por eso su complejidad final es $O(n \log n)$.

Esta complejidad se mantiene en el mejor, promedio y peor caso. Esto ocurre porque Merge Sort siempre divide el arreglo en mitades y siempre realiza las mezclas, sin importar si los datos venían ordenados, invertidos o desordenados. Su principal desventaja es que necesita memoria adicional para la etapa de mezcla.

3.3.3. Merge Sort optimizado con threshold

La versión optimizada mantiene la misma idea de Merge Sort, pero agrega una mejora práctica. Cuando el subarreglo es muy pequeño, seguir dividiendo puede no ser conveniente, porque se hacen muchas llamadas recursivas para ordenar muy pocos elementos.

Para evitar eso se usa un *threshold*, es decir, un tamaño límite. Si el subarreglo tiene un tamaño menor o igual a ese límite, se ordena directamente con Insertion Sort. Si el subarreglo es más grande, el algoritmo continúa como Merge Sort normal.

Algorithm 6 Merge Sort optimizado con threshold ▷ $V[left..right]$

```

1: procedure MERGESORTOPTIMIZED( $V, left, right, threshold, comp\_f$ )
2:   if  $right - left + 1 \leq threshold$  then
3:     InsertionSort( $V, left, right, comp\_f$ )
4:   return
5:   if  $left \geq right$  then
6:     return
7:    $mid \leftarrow left + (right - left)/2$ 
8:   MergeSortOptimized( $V, left, mid, threshold, comp\_f$ )
9:   MergeSortOptimized( $V, mid + 1, right, threshold, comp\_f$ )
10:  Merge( $V, left, mid, right, comp\_f$ )
11: 
```

▷ Caso optimizado para subarreglos pequeños

▷ Caso base: subarreglo vacío o de un elemento

▷ Dividir izquierda

▷ Dividir derecha

▷ Conquistar: mezclar

Esta optimización no cambia la complejidad teórica principal del algoritmo, que sigue siendo $O(n \log n)$. Sin embargo, puede mejorar los tiempos reales de ejecución, porque Insertion Sort suele funcionar bien en arreglos pequeños.

3.3.4. Análisis experimental del umbral

Para evaluar esta mejora, primero se comparó Insertion Sort con Merge Sort clásico en tamaños pequeños. La idea era comprobar si tenía sentido usar Insertion Sort como apoyo en subarreglos cortos.³

En la primera comparación (Figura 2) se observa que Insertion Sort puede ser competitivo cuando la entrada es pequeña. Sin embargo, cuando el tamaño crece, Merge Sort empieza a tener mejores tiempos. Esto muestra que Insertion Sort conviene solo para segmentos pequeños, no como algoritmo principal para entradas grandes.

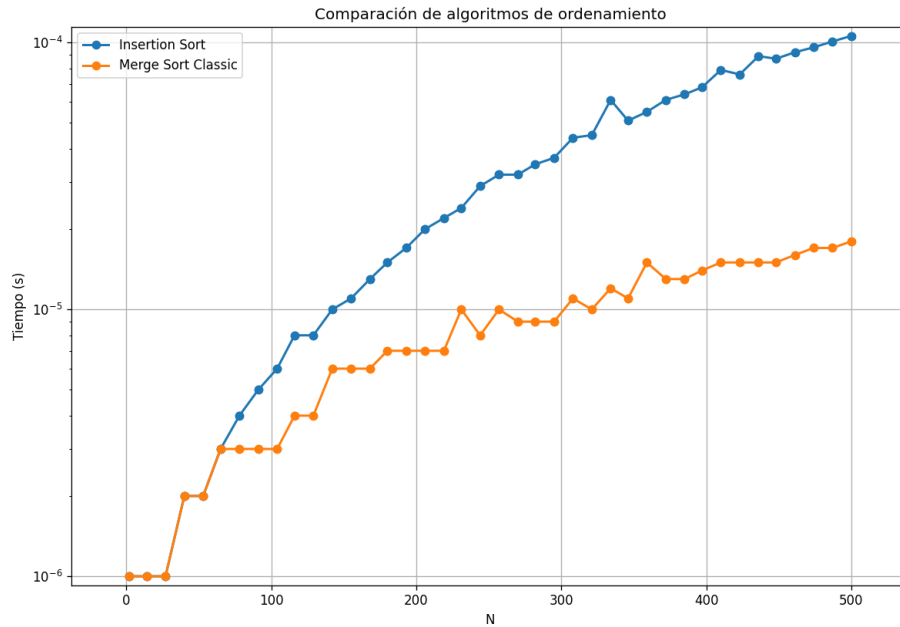


Figura 2. Comparación entre Insertion Sort y Merge Sort clásico para tamaños de entrada pequeños (escala logarítmica).

Luego se comparó Merge Sort clásico con distintas versiones optimizadas usando varios valores de *threshold*. En la Figura 3 se puede ver que las versiones optimizadas tienden a estar por debajo de la versión clásica, lo que indica menores tiempos de ejecución.

³El experimento que arroja los resultados mostrados en esta sección puede ser replicado compilando [este commit](#) del repositorio del proyecto.

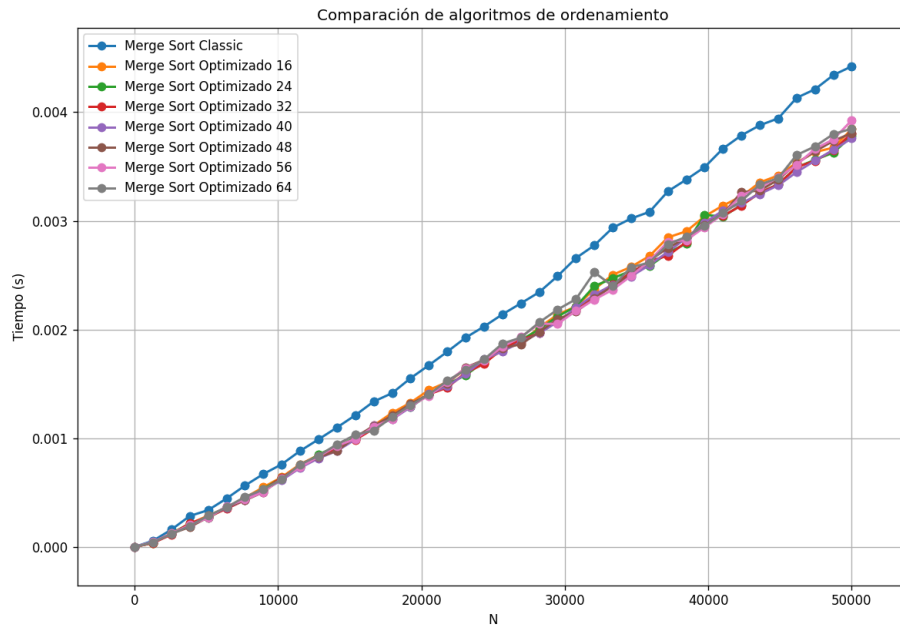


Figura 3. Comparación entre Merge Sort clásico y versiones optimizadas con distintos umbrales.

A partir de estas pruebas se escogió el umbral usado en la versión optimizada. La elección se hizo observando los resultados experimentales, buscando un punto en que se redujera la recursión sin hacer que Insertion Sort trabajara con subarreglos demasiado grandes.

Finalmente, se comparó Merge Sort clásico con Merge Sort optimizado usando el umbral seleccionado. En la Figura 4 se observa que ambos algoritmos mantienen una forma de crecimiento parecida, pero la versión optimizada obtiene tiempos menores dentro del rango evaluado.

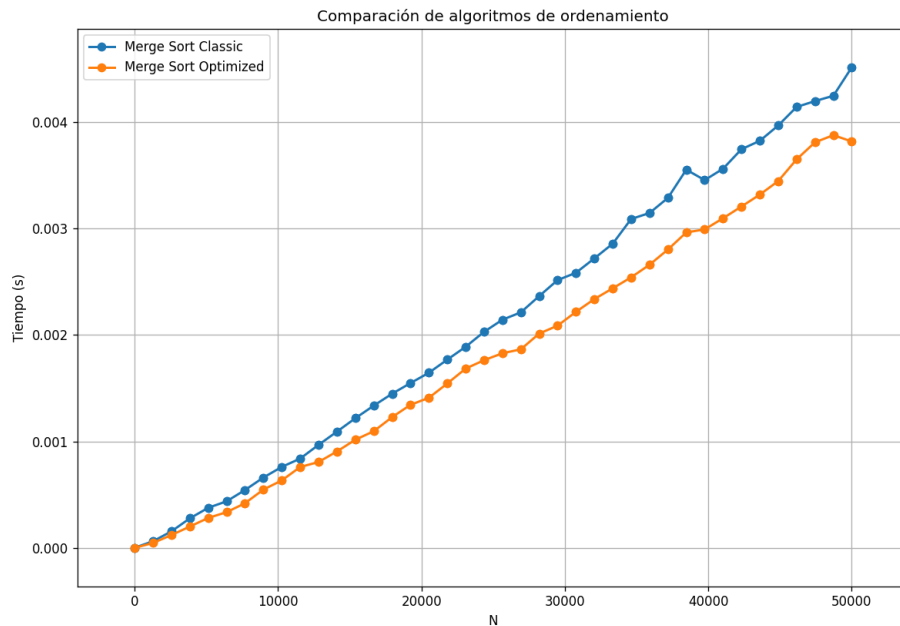


Figura 4. Comparación final entre Merge Sort clásico y Merge Sort optimizado.

En conjunto, los resultados muestran que usar un *threshold* puede ser útil en la práctica. La mejora no cambia la complejidad $O(n \log n)$, pero sí ayuda a reducir tiempo de ejecución al evitar recursión innecesaria en subarreglos pequeños.

3.3.5. Comentario general

En conclusión, Merge Sort se puede entender a partir de tres ideas simples. Primero, divide el arreglo en mitades. Segundo, ordena esas mitades de forma recursiva. Tercero, las une con la función *merge*. La versión clásica mantiene un comportamiento estable de $O(n \log n)$ en todos los casos. La versión optimizada conserva esa misma idea, pero usa Insertion Sort en subarreglos pequeños para mejorar el rendimiento práctico.

3.4. Búsqueda Binaria Recursiva

La búsqueda binaria recursiva es una variante de la búsqueda binaria iterativa que resuelve el problema mediante la estrategia de divide y vencerás. Su aplicación requiere que los datos se encuentren previamente ordenados, ya que en cada paso el algoritmo decide en qué mitad del arreglo continuar a partir de una comparación con el elemento central.

En el sistema desarrollado del proyecto 1 ya se contaba con búsqueda secuencial y búsqueda binaria iterativa. En esta tarea se incorpora la versión recursiva, con el objetivo de analizar su comportamiento, y compararlo con los algoritmos de búsqueda del proyecto anterior, y los algoritmos de búsqueda implementados en el proyecto actual.

Su funcionamiento consiste en comparar el elemento situado en la posición media del subarreglo actual. Si el elemento medio coincide con el valor buscado, la búsqueda termina exitosamente. Si el valor buscado es menor, la búsqueda continúa de manera recursiva en la mitad izquierda. Si es mayor, continúa en la mitad derecha. De esta forma, en cada llamada recursiva se deshace aproximadamente la mitad de los elementos del arreglo a evaluar.

Algorithm 7 Búsqueda binaria recursiva de jugadores $\triangleright V[beg..end]$

```

1: procedure BINARYSEARCHRECURSIVE( $V, beg, end, x, comp\_f$ )
2:   if  $beg > end$  then
3:     return  $-1$ 
4:    $mid \leftarrow beg + \frac{end - beg}{2}$ 
5:    $cmp \leftarrow comp\_f(\&V[mid], x)$ 
6:   if  $cmp = 0$  then
7:     return  $mid$ 
8:   if  $cmp > 0$  then
9:     return BINARYSEARCHRECURSIVE( $V, beg, mid - 1, x, comp\_f$ )
10:  return BINARYSEARCHRECURSIVE( $V, mid + 1, end, x, comp\_f$ )

```

3.4.1. Análisis de complejidad

En cada llamada recursiva se genera un único subproblema de tamaño aproximado $\frac{n}{2}$, mientras que el trabajo adicional fuera de la llamada recursiva consiste en calcular la posición central y realizar una o dos comparaciones, obteniendo lo siguiente:

$$T(n) = T(n/2) + O(1)$$

Esta recurrencia se ajusta a la forma general del Teorema maestro presentada en la sección 9.2, entonces:

$$T(n) = O(\log n)$$

Por tanto la complejidad temporal de la búsqueda binaria recursiva es:

- Mejor caso: $O(1)$, cuando el elemento buscado se encuentra en la posición media.

- Caso promedio: $O(\log n)$
- Peor caso: $O(\log n)$

En comparación con la versión iterativa, la complejidad temporal asintótica es la misma. La diferencia principal está en que la versión recursiva utiliza la pila de llamadas del sistema, lo que genera una carga adicional, aunque sin modificar el crecimiento del algoritmo.

3.4.2. Comentario general

Desde el punto de vista teórico, se espera que este algoritmo implementado en su versión recursiva, mantenga un comportamiento claramente superior al de búsqueda secuencial a medida que va aumentando el tamaño de entrada, debido a su crecimiento logarítmico. Experimentalmente, sus tiempos deberían ser muy cercanos a los de su versión iterativa, ya que ambas aplican la misma idea de reducción por mitades, diferenciándose solo en la forma que fueron implementadas.

3.5. Búsqueda Binaria por Rango

La búsqueda binaria por rango es una variante de la búsqueda binaria que permite determinar el intervalo de posiciones en el que aparece un valor dentro de un arreglo ordenado, o incluso un segmento de elementos cuyos valores se encuentran dentro de un rango establecido.

En este proyecto se implementaron dos variantes complementarias de esta idea:

1. Búsqueda por valor exacto: recibe un único valor y determina la primera y última posición en la que aparece dentro del arreglo.
2. Búsqueda por intervalo: recibe un valor mínimo y un valor máximo, y determina el bloque de elementos cuyos valores están entre ambos límites.

Esta distinción resulta útil en el contexto del sistema de deportistas, ya que múltiples jugadores pueden compartir el mismo puntaje. Por eso mismo, no basta con solo encontrar una sola coincidencia, ya que, en muchos casos interesa recuperar todas las concurrencias asociadas a un valor exacto, o bien todos los deportistas cuyo puntaje pertenezcan a un intervalo definido.

3.5.1. Búsqueda por valor exacto: `first` y `last`

La primera variante consiste en localizar todas las ocurrencias de un valor exacto dentro de un arreglo ordenado. Para ello se implementan dos funciones auxiliares:

- `first`: encuentra la primera posición en la que aparece el valor buscado.
- `last`: encuentra la última posición del valor buscado.

La idea consiste en ejecutar dos búsquedas binarias modificadas. En ambos casos, cuando se encuentra una coincidencia, el algoritmo no termina inmediatamente, sino que continúa explorando hacia la izquierda o hacia la derecha para hallar el extremo correspondiente. De esta manera `first` sigue buscando hacia la izquierda mientras existan coincidencias anteriores, y `last` sigue buscando hacia la derecha mientras existan coincidencias posteriores. Una vez obtenido ambos índices, el rango completo de ocurrencias queda determinado por el intervalo `[first, last]`.

3.5.2. Búsqueda por intervalo: `lower_bound` y `upper_bound`

La segunda variante corresponde a la búsqueda de todos los elementos cuyos valores se encuentran dentro de un rango `[min, max]`. En este caso no se busca un valor exacto, sino los límites del segmento válido dentro del arreglo ordenado.

Para ello se implementan las siguientes dos funciones auxiliares:

- `lower_bound` : determina la primera posición cuyo valor es mayor o igual que el límite inferior.
- `upper_bound` : determina la última posición cuyo valor es menor o igual que el límite superior.

Estas funciones permiten identificar directamente el subarreglo que satisface la condición del rango, evitando recorrer elementos innecesarios. Si el índice retornado por `lower_bound` supera el retornado por `upper_bound`, entonces el rango buscado no contiene elementos válidos.

3.5.3. Idea general en pseudocódigo

A continuación se presentan pseudocódigos que describen de manera abstracta el comportamiento de estas variantes de búsqueda binaria por rango. Para observar su implementación exacta, puede consultarse el código fuente disponible en el repositorio.

En la búsqueda por valor exacto, el proceso es:

Algorithm 8 Rango de apariciones $\triangleright V[0..n - 1]$

```

1: procedure SEARCHRANGE( $V, x$ )
2:    $first \leftarrow \text{FIRSTOCURRENCE}(V, x)$ 
3:   if  $first = -1$  then
4:     return  $\emptyset$ 
5:    $last \leftarrow \text{LASTOCURRENCE}(V, x)$ 
6:   return  $[first, last]$ 

```

En la búsqueda por intervalo, el procedimiento es:

Algorithm 9 Rango de valores entre mínimo y máximo $\triangleright V[0..n - 1]$

```

1: procedure SEARCHRANGEBETWEEN( $V, min, max$ )
2:    $first \leftarrow \text{LOWERBOUND}(V, min)$ 
3:    $last \leftarrow \text{UPPERBOUND}(V, max)$ 
4:   if  $first > last$  then
5:     return  $\emptyset$ 
6:   return  $[first, last]$ 

```

En ambos casos, el resultado final no es un único índice, sino un intervalo de posiciones que representa todas las coincidencias con el valor o segmento solicitado.

3.5.4. Análisis de complejidad

Cada una de las funciones auxiliares (`first`, `last`, `lower_bound`, `upper_bound`) realiza una búsqueda binaria sobre un arreglo ordenado, por lo que su complejidad es:

$$O(\log n)$$

Como cada consulta requiere dos búsquedas de este tipo, el costo total sigue siendo:

$$O(\log n)$$

ya que la suma de dos términos logarítmicos mantiene el mismo orden de crecimiento.

Por tanto, ambas variantes:

- Mejor caso: $O(\log n)$

- Caso promedio: $O(\log n)$
- Peor caso: $O(\log n)$

Esto representa una mejora importante respecto de una solución lineal, que requeriría recorrer potencialmente todo el arreglo para identificar el conjunto de elementos buscados.

3.5.5. Comentario general

La búsqueda binaria por rango aumenta la utilidad de la búsqueda binaria clásica, permitiendo recuperar conjuntos de elementos en lugar de una sola coincidencia. En este proyecto, su implementación en dos variantes diferenciadas, una basada en `first` y `last` para valores exactos, y otra basada en `lower_bound` y `upper_bound` para intervalos, permite resolver consultas más expresivas sobre los puntajes de los deportistas sin abandonar la eficiencia logarítmica del enfoque divide y vencerás.

3.6. Búsqueda Exponencial

La búsqueda exponencial es un algoritmo híbrido de búsqueda que combina una etapa inicial de expansión rápida del rango de búsqueda con una etapa posterior de búsqueda binaria. Su principal utilidad aparece cuando trabaja con arreglos ordenados y se desea localizar eficientemente un elemento sin conocer de antemano una zona acotada en la que este pueda encontrarse.

La idea central del algoritmo consiste en aumentar exponencialmente el índice inspeccionado hasta encontrar un valor mayor o igual al objetivo, o bien hasta superar el tamaño del arreglo. Una vez determinado ese intervalo, se aplica búsqueda binaria únicamente sobre dicho subarreglo. De esta manera, el algoritmo evita buscar desde el principio sobre todo el arreglo y acota rápidamente la región relevante.

En este proyecto, la búsqueda exponencial se utiliza sobre arreglos ordenados por `id`, de modo que la comparación entre elementos mantenga la propiedad necesaria para descartar regiones completas del arreglo.

3.6.1. Funcionamiento general

El procedimiento puede describirse en dos etapas:

1. Expansión exponencial del rango: Se comienza revisando el primer elemento del arreglo. Si no coincide con el valor buscado, se inspeccionan posiciones que crecen como potencias de dos: `1, 2, 4, 8, 16, ...` hasta encontrar una posición cuyo valor sea mayor o igual al objetivo, o hasta alcanzar el final del arreglo.
2. Búsqueda binaria en el intervalo encontrado: Una vez acotado el rango donde potencialmente se encuentra el elemento, se ejecuta una búsqueda binaria clásica sobre el subarreglo.

Este enfoque resulta eficiente porque la fase exponencial reduce rápidamente el espacio de búsqueda, mientras que la fase binaria aprovecha esa cota para completar la localización del elemento.

La parte que destaca de este algoritmo es la expansión exponencial. En lugar de recorrer el arreglo elemento a elemento, el algoritmo duplica progresivamente la posición inspeccionada, lo que le permite acercarse muy rápido a la zona donde podría estar el valor buscado. Una vez encontrado el límite superior, el problema se reduce a una búsqueda binaria convencional en un intervalo acotado.

En este sentido, este algoritmo se puede interpretar como una optimización para situaciones en las que el valor buscado podría hallarse relativamente cerca del inicio del arreglo, pero sin perder eficiencia cuanto más lejos este el elemento.

Algorithm 10 Búsqueda exponencial $\triangleright V[0..n-1]$

```

1: procedure EXPONENTIALSEARCH( $V, n, x, comp\_f$ )
2:   if  $n \leq 0$  then
3:     return  $-1$ 
4:   if  $comp\_f(\&V[0], x) = 0$  then
5:     return  $0$ 
6:    $bound \leftarrow 1$ 
7:   while  $bound < n$  and  $comp\_f(\&V[bound], x) < 0$  do
8:      $bound \leftarrow bound \cdot 2$ 
9:    $beg \leftarrow \frac{bound}{2}$ 
10:  if  $bound < n$  then
11:     $end \leftarrow bound$ 
12:  else
13:     $end \leftarrow n - 1$ 
14:  return BINARYSEARCH( $V, beg, end, x, comp\_f$ )

```

3.6.2. Análisis de complejidad

La complejidad temporal de la búsqueda exponencial se obtiene sumando el costo de sus dos etapas, donde la primera corresponde a la fase exponencial, donde cada en cada iteración el índice se duplica, lo que corresponde a orden de $\log n$, y la segunda corresponde a la fase binaria que se ejecuta una vez determinado el intervalo, que también cuesta $\log n$.

Por lo tanto, el costo total es:

$$O(\log n)$$

De esta forma, la complejidad temporal queda:

- Mejor caso: $O(1)$, si el elemento se encuentra en primera posición.
- Caso promedio: $O(\log n)$
- Peor caso: $O(\log n)$

Aunque comparte el mismo orden de complejidad que búsqueda binaria, la búsqueda exponencial incorpora una fase previa adicional. Sin embargo, esa fase también crece logarítmicamente, por lo que el comportamiento asintótico no se ve alterado.

3.6.3. Comentario general

La búsqueda exponencial representa una extensión de la búsqueda binaria para escenarios en los que no se conoce de antemano una región acotada del arreglo donde podría encontrarse el elemento buscado. Su ventaja principal se encuentra en que acota rápidamente el intervalo antes de aplicar búsqueda binaria, manteniendo en todo momento una complejidad logarítmica. En el contexto de este proyecto, constituye una alternativa eficiente sobre arreglos ordenados por `id`, y resulta interesante para comparar cómo distintas variantes de búsqueda basadas en divide y vencerás alcanzan órdenes de crecimiento similares mediante estrategias operacionales distintas.

3.7. Búsqueda por Interpolación

La búsqueda por interpolación es una variante de la búsqueda binaria diseñada para trabajar sobre arreglos ordenados cuyos valores presentan una distribución aproximadamente uniforme. A diferencia de la búsqueda binaria, que siempre inspecciona el elemento central del intervalo actual, la búsqueda por interpolación intenta estimar directamente una posición más cercana al valor buscado a partir de la relación numérica entre los extremos del intervalo y el objetivo.

La intuición detrás del algoritmo es similar a la búsqueda de una palabra en un diccionario o de un nombre en una guía telefónica: en vez de abrir siempre en la mitad exacta, se estima una posición probable según el valor que se desea encontrar. Si la distribución de los datos es uniforme o cercana a uniforme, esta estimación puede acercarse bastante a la posición real del elemento, reduciendo aún más el número de comparaciones necesarias.

En este proyecto, la búsqueda por interpolación se aplica sobre arreglos ordenados por `id`, ya que esta clave posee una naturaleza numérica y ordenada que permite calcular una posición estimada dentro del subarreglo actual.

3.7.1. Funcionamiento general

El algoritmo trabaja sobre un intervalo delimitado por dos posiciones, `beg` (izquierda) y `end` (derecha). En cada paso calcula una posición estimada `pos` mediante una fórmula de interpolación lineal, basada en:

- El valor buscado
- El valor en el extremo izquierdo
- El valor en el extremo derecho

Luego compara el valor ubicado en `pos` con el objetivo de si coincide, la búsqueda termina; si el objetivo es menor, continúa en la parte izquierda; si es mayor, continúa en la parte derecha.

De este modo, el algoritmo también reduce progresivamente el espacio de búsqueda, pero lo hace intentando aproximarse mejor a la posición esperada del valor buscado.

La característica que distingue a este algoritmo es el cálculo de la posición estimada. Mientras la búsqueda binaria divide el intervalo en mitades exactas, la búsqueda por interpolación intenta aprovechar la magnitud relativa del valor buscado para ubicar una posición más prometedora.

Si los valores del arreglo están distribuidos de manera uniforme, la posición calculada puede quedar muy cerca de la real, haciendo que el número de pasos necesarios sea muy bajo. Sin embargo, cuando la distribución de los datos es irregular o presenta acumulaciones, la estimación pierde precisión y el comportamiento del algoritmo puede verse afectado.

Por esta razón, el rendimiento de la búsqueda por interpolación depende no solo del tamaño del arreglo, sino también de la forma en que se distribuyen sus claves.

3.7.2. Análisis de complejidad

El comportamiento teórico de la búsqueda por interpolación depende de la distribución de los datos:

- Mejor caso: $O(1)$, si la primera estimación resulta ser la posición del valor buscado.
- Caso promedio: $O(\log \log n)$, bajo una distribución uniforme o aproximadamente uniforme.
- Peor caso: $O(n)$

Esta diferencia respecto de la búsqueda binaria es importante. Aunque la búsqueda por interpolación puede superar en rendimiento a la búsqueda

Algorithm 11 Búsqueda por interpolación ▷ $V[beg..end]$

```

1: procedure INTERPOLATIONSEARCH( $V, beg, end, target\_id$ )
2:   while  $beg \leq end$  and  $target\_id \geq V[beg].id$  and  $target\_id \leq V[end].id$  do
3:     if  $V[beg].id = V[end].id$  then
4:       if  $V[beg].id = target\_id$  then
5:         return  $beg$ 
6:       return  $-1$ 
7:      $numerator \leftarrow (target\_id - V[beg].id) \cdot (end - beg)$ 
8:      $denominator \leftarrow V[end].id - V[beg].id$ 
9:      $pos \leftarrow beg + \lfloor \frac{numerator}{denominator} \rfloor$ 
10:    if  $pos < beg$  then
11:       $pos \leftarrow beg$ 
12:    else if  $pos > end$  then
13:       $pos \leftarrow end$ 
14:    if  $V[pos].id = target\_id$  then
15:      return  $pos$ 
16:    if  $V[pos].id < target\_id$  then
17:       $beg \leftarrow pos + 1$ 
18:    else
19:       $end \leftarrow pos - 1$ 
20:  return  $-1$ 

```

binaria en situaciones favorables, no ofrece la misma estabilidad teórica, dado que su peor caso puede degradar esta estabilidad considerablemente.

3.7.3. Comentario general

La búsqueda por interpolación constituye una variante más especializada de las búsquedas sobre arreglos ordenados. Su fortaleza corresponde a como explota la distribución numérica de los datos para estimar de manera más precisa la posición del elemento buscado. En arreglos uniformemente distribuidas, esta estrategia puede ofrecer un rendimiento muy superior al de búsqueda binaria, aunque dependa de la distribución numérica.

4. Comparación de algoritmos de ordenamiento

Con el fin de obtener una visión global del rendimiento de todos los algoritmos implementados, se comparan a continuación los resultados experimentales de los algoritmos de Divide y Vencerás introducidos en este proyecto (Quick Sort, Merge Sort y Merge Sort Optimizado) frente a los cuatro algoritmos iterativos estudiados en el proyecto anterior: Bubble Sort, Insertion Sort, Selection Sort y Cocktail Shaker Sort. La comparación se realiza bajo los mismos tres escenarios de distribución de datos utilizados a lo largo del informe.

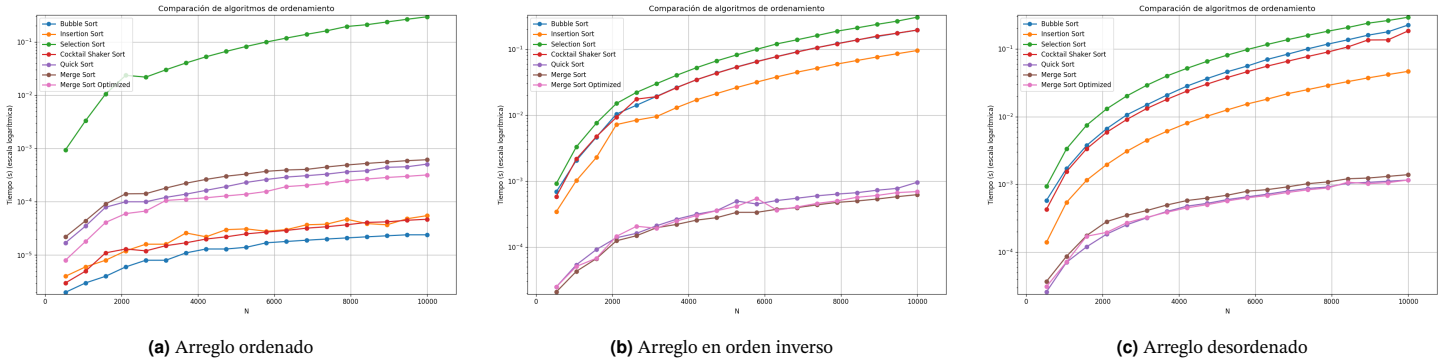


Figura 5. Comparación de algoritmos de ordenamiento iterativos y Divide y Vencerás bajo los tres escenarios de distribución de datos.

Arreglo invertido y desordenado

En el escenario de orden inverso (Figura 5b) y en el caso promedio con datos aleatorios (Figura 5c), la brecha entre los algoritmos iterativos y los de Divide y Vencerás es notoria. Quick Sort, Merge Sort y Merge Sort Optimizado se agrupan en el extremo inferior del gráfico, separados por aproximadamente dos órdenes de magnitud respecto a Selection Sort y un orden respecto a Bubble Sort y Cocktail Shaker Sort. Este comportamiento es consistente con la superioridad asintótica de $O(n \log n)$ frente a $O(n^2)$ para tamaños de entrada crecientes. Dentro del grupo de Divide y Vencerás, los tres algoritmos exhiben tiempos de ejecución muy similares entre sí en ambos escenarios.

Arreglo ordenado: una ventaja de los algoritmos iterativos

El escenario de datos ya ordenados (Figura 5a) revela un resultado llamativo: los algoritmos iterativos con detección temprana de terminación (Bubble Sort, Insertion Sort y Cocktail Shaker Sort) superan en velocidad a los algoritmos de Divide y Vencerás. Esto se debe a que, cuando el arreglo ya está ordenado, dichos algoritmos alcanzan su mejor caso de $O(n)$, realizando únicamente un recorrido lineal sin efectuar intercambios o inserciones. Quick Sort y Merge Sort, en cambio, continúan ejecutando su esquema de división recursiva independientemente del estado del arreglo, manteniéndose en $O(n \log n)$.

La única excepción entre los iterativos es Selection Sort, que no posee mecanismo de detección de ordenamiento previo y mantiene su complejidad $O(n^2)$ incluso en el mejor caso, tal como se analizó en el proyecto anterior.

5. Comparación de algoritmos de búsqueda

Con el fin de obtener una visión global del rendimiento de los algoritmos de búsqueda implementados, se comparan a continuación los resultados experimentales de búsqueda secuencial, búsqueda binaria iterativa, búsqueda binaria recursiva, búsqueda exponencial, búsqueda por interpolación y búsqueda binaria por rango. A diferencia del proyecto anterior, donde la referencia principal era la búsqueda secuencial y búsqueda binaria iterativa, en esta etapa se incorporan variantes basadas en la estrategia de Divide y Vencerás, permitiendo contrastar experimentalmente comportamientos lineales, logarítmicos y, en el caso de la interpolación, dependientes de la distribución de los datos.

El análisis experimental se realizó sobre tamaños de entrada crecientes, evaluados en 20 puntos distribuidos a lo largo del rango considerado. Para cada tamaño se efectuaron 100 mediciones, promediando luego los tiempos obtenidos. En los algoritmos más rápidos se repitió internamente la operación de búsqueda varias veces por medición, con el fin de obtener tiempos observables incluso en escalas muy pequeñas. Para mantener una comparación estable entre los algoritmos de búsqueda exacta, se utilizó el campo `id` como clave en búsqueda secuencial, búsqueda binaria iterativa, búsqueda binaria recursiva, búsqueda exponencial e interpolación. En cambio, la búsqueda binaria por rango se evaluó sobre el campo `score`, ya que en ese caso se busca recuperar un conjunto de elementos y no una única coincidencia.

Asimismo, dado que todos los algoritmos considerados, salvo la búsqueda secuencial, requieren arreglos previamente ordenados para funcionar correctamente, el costo de ordenamiento se separó del costo de búsqueda. De este modo, los tiempos reportados en las gráficas representan únicamente el comportamiento de los algoritmos de búsqueda y no el tiempo de preparación de la entrada.

Peor caso

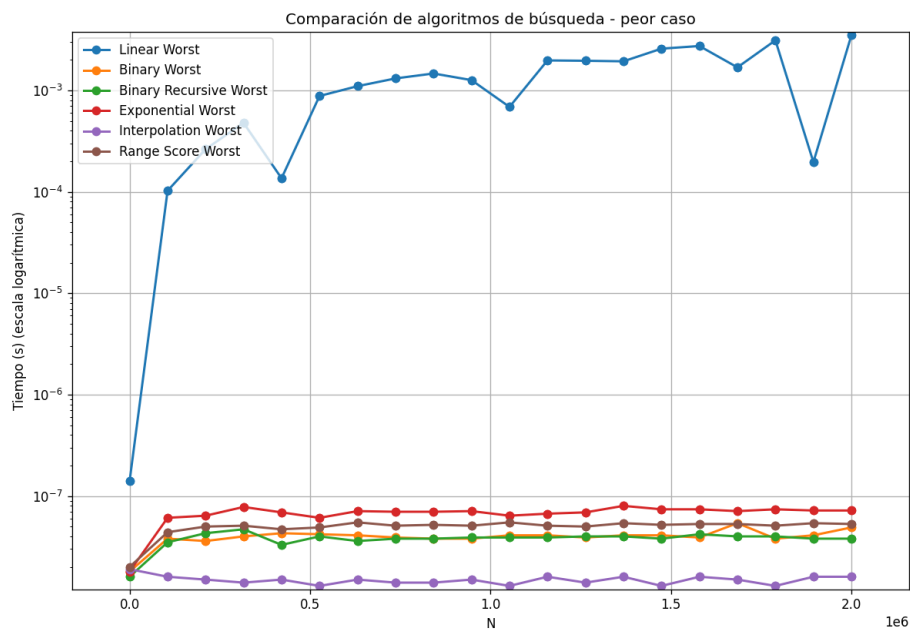


Figura 6. Comparación de los peores casos de algoritmos de búsqueda (escala logarítmica)

En el peor caso, la búsqueda secuencial se separa claramente del resto de los algoritmos. Su curva crece con mucha mayor rapidez y domina ampliamente la escala temporal a medida que aumenta el tamaño de entrada. Este comportamiento es consistente con su complejidad lineal $O(n)$, ya que en este escenario el algoritmo debe recorrer el arreglo completo antes de concluir que el identificador buscado no se encuentra presente.

En contraste, la búsqueda binaria iterativa, la búsqueda binaria recursiva, la búsqueda exponencial y la búsqueda binaria por rango se agrupan en la parte inferior del gráfico, con tiempos muy próximos entre sí y claramente inferiores a los de la búsqueda secuencial. Esto coincide con

sus complejidades logarítmicas y confirma experimentalmente la ventaja de descartar grandes porciones del arreglo en cada paso, en lugar de recorrerlo de manera lineal.

Dentro de este grupo, la búsqueda binaria iterativa y la búsqueda binaria recursiva presentan comportamientos prácticamente equivalentes, lo que resulta esperable dado que ambas aplican la misma estrategia de reducción por mitades y solo difieren en el uso de iteración o recursión. La búsqueda exponencial, por su parte, también se mantiene en esta misma escala, confirmando que la fase inicial de expansión del intervalo no altera sustancialmente su comportamiento asintótico.

La búsqueda binaria por rango sobre `score` también exhibe tiempos bajos y estables. Aunque requiere determinar ambos extremos del rango buscado, su comportamiento continúa siendo logarítmico, por lo que permanece muy por debajo de la búsqueda secuencial incluso en entradas grandes.

Caso promedio

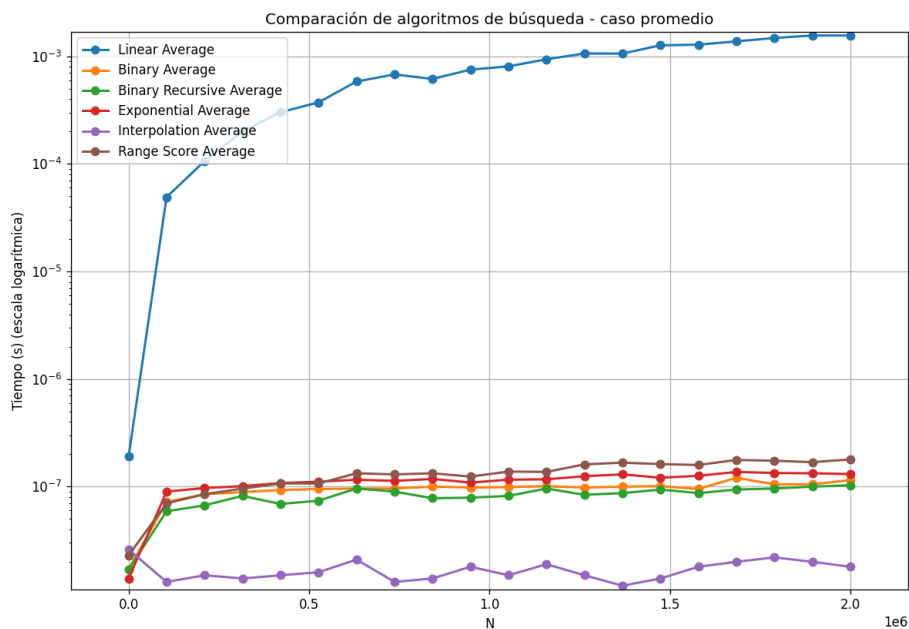


Figura 7. Comparación de los casos promedio de algoritmos de búsqueda (escala logarítmica)

En el caso promedio se observa nuevamente una separación entre la búsqueda secuencial y el resto de los algoritmos. Aunque en promedio la búsqueda lineal no necesita necesariamente recorrer todo el arreglo, su crecimiento sigue siendo mucho mayor que el de las variantes basadas en divide y vencerás.

Las búsquedas binaria iterativa y binaria recursiva vuelven a mostrar un comportamiento muy cercano, ratificando que ambas comparten la misma eficiencia asintótica. La búsqueda exponencial también se mantiene dentro de esta misma zona del gráfico, con tiempos comparables a los de la búsqueda binaria. Esto permite observar que, en la práctica, las diferencias entre estas variantes son menores que la distancia que las separa de la búsqueda secuencial.

La búsqueda binaria por rango presenta tiempos algo superiores a los de la búsqueda binaria simple, lo cual es razonable, pues en vez de localizar una sola posición debe determinar los límites del segmento relevante dentro del arreglo. Aun así, su comportamiento sigue siendo considerablemente más eficiente que el de la búsqueda lineal y conserva la misma tendencia logarítmica general.

En conjunto, los resultados del caso promedio confirman que, para arreglos ordenados y de gran tamaño, las estrategias de búsqueda basadas en

reducción del espacio de búsqueda superan ampliamente a la exploración secuencial, tanto en teoría como en práctica.

El caso particular de la búsqueda por interpolación

La búsqueda por interpolación merece una observación separada, ya que en ambas gráficas aparece con tiempos excepcionalmente bajos en comparación con las demás variantes de búsqueda exacta. Este resultado se explica porque el análisis se realizó utilizando `id` como clave, y dicha clave presenta una estructura numérica favorable para la estimación de posiciones que realiza este algoritmo.

En términos teóricos, esta observación es coherente con su comportamiento esperado sobre distribuciones aproximadamente uniformes, donde el caso promedio puede acercarse a $O(\log \log n)$. Sin embargo, esto no debe confundirse con su peor caso teórico, que sigue siendo $O(n)$. En otras palabras, la interpolación puede resultar extraordinariamente eficiente cuando la distribución de los datos la favorece, pero su comportamiento es más sensible a la estructura de la entrada que el de la búsqueda binaria o la exponencial.

Por esta razón, sus resultados deben interpretarse como una evidencia de muy buen rendimiento práctico bajo las condiciones particulares del experimento.

6. Aplicación práctica

6.1. Interfaz de línea de comandos

Con el fin de integrar los algoritmos implementados al sistema desarrollado en el Proyecto 1, se rediseñó la interfaz de línea de comandos para permitir invocar cada funcionalidad de forma directa, sin depender de interacción paso a paso. Para ello se utilizó la función estándar `getopt_long()`, disponible en `<getopt.h>`, la cual permite definir tanto banderas cortas (por ejemplo, `-s`) como sus equivalentes en formato largo (por ejemplo, `-sort`). Esto simplifica el procesamiento de argumentos y hace posible exponer cada operación del sistema como una acción independiente parametrizable desde la terminal.

De esta forma, el programa admite operaciones de generación, lectura, ordenamiento, búsqueda, selección, ranking, búsqueda por puntaje exacto, búsqueda por rango y ejecución de experimentos. Algunos ejemplos de uso se muestran a continuación:

```
1 ./build/conquer.out -g 1000 -t shuffled
2 ./build/conquer.out -r
3 ./build/conquer.out -s -a quick -c score
4 ./build/conquer.out -f interpolation -i 25
5 ./build/conquer.out -j 3
6 ./build/conquer.out -p 10
7 ./build/conquer.out -q 80.0
8 ./build/conquer.out -R -m 80.0 -M 95.0
9 ./build/conquer.out -e
```

Para la generación de datos se utiliza la bandera `-g` acompañada de un número de elementos, junto con `-t` para indicar el tipo de generación. Los modos soportados son `sorted`, `inverse` y `shuffled`, permitiendo construir arreglos ordenados, invertidos o aleatorios según las necesidades del análisis experimental.

```
1 ./build/conquer.out -g <cantidad> -t <sorted|inverse|shuffled>
```

Para visualizar el archivo CSV actualmente almacenado se emplea la bandera `-r` (`-read`), la cual carga y muestra el contenido actual del sistema.

```
1 ./build/conquer.out -r
```

Para ordenar el conjunto de deportistas se expone la bandera `-s` (`-sort`), que puede acompañarse de `-a` para indicar el algoritmo y de `-c` para el criterio de comparación. Los algoritmos admitidos incluyen las variantes implementadas en el proyecto, mientras que los criterios corresponden a los distintos campos del registro del deportista. Si la opción `-a` se omite, el sistema utiliza **Quick Sort** como algoritmo predeterminado; si `-c` se omite, se utiliza el campo `id` como criterio por defecto.

```
1 ./build/conquer.out -s [-a <algoritmo>] [-c <criterio>]
```

Para la búsqueda exacta de un deportista por identificador se utiliza la bandera `-f`, junto con `-i` para indicar el `id` buscado. Esta funcionalidad admite distintas variantes de búsqueda exacta, incluyendo búsqueda secuencial, búsqueda binaria iterativa, búsqueda binaria recursiva, búsqueda exponencial e interpolación. Si no se especifica un algoritmo explícitamente, el sistema utiliza **búsqueda por interpolación** como algoritmo predeterminado.

```
1 ./build/conquer.out -f [algoritmo-busqueda] -i <id>
```

Para obtener el k -ésimo mejor deportista se emplea la bandera `-j` seguida del valor entero k . Esta funcionalidad se implementa mediante **Quick Select**, utilizando el puntaje como criterio de selección.

```
1 ./build/conquer.out -j <k>
```

El ranking de los N mejores deportistas se obtiene con la bandera `-p`. Internamente, el sistema utiliza **Quick Select** para aislar el segmento correspondiente a los mejores puntajes y luego ordena dicho segmento para presentarlo como ranking final.

```
1 ./build/conquer.out -p <N>
```

Para recuperar todos los deportistas que tengan un puntaje exacto se utiliza la bandera `-q` seguida del valor buscado. En este caso, el sistema ordena previamente los datos por puntaje y luego aplica una variante de **búsqueda binaria por rango**, identificando la primera y la última ocurrencia del valor solicitado.

```
1 ./build/conquer.out -q <score>
```

Para consultar los deportistas cuyo puntaje se encuentre entre un mínimo y un máximo se utiliza la bandera `-R` acompañada de `-m` y `-M`. El sistema ordena primero el arreglo por puntaje y posteriormente aplica **búsqueda binaria por rango**, determinando el subarreglo comprendido entre el primer valor válido y el último valor válido dentro del intervalo $[min, max]$.

```
1 ./build/conquer.out -R -m <min> -M <max>
```

Para la ejecución de experimentos se utiliza la bandera `-e`. Esta opción despliega un menú interno desde el cual el usuario puede escoger si desea ejecutar el experimento de algoritmos de ordenamiento o el de algoritmos de búsqueda.

```
1 ./build/conquer.out -e
```

Finalmente, la bandera `-h` permite desplegar la ayuda general del programa, incluyendo la sintaxis disponible y los algoritmos predeterminados definidos para ciertas operaciones.

```
1 ./build/conquer.out -h
```

Cabe señalar que los argumentos escritos entre corchetes `[]` son opcionales. Cuando el usuario omite alguno de ellos, el programa recurre a valores predeterminados seleccionados por los autores, de acuerdo con los resultados obtenidos en el análisis teórico y experimental.

6.2. Justificación de la elección de algoritmos

6.2.1. Algoritmo de ordenamiento: Quick Sort

Aunque los resultados experimentales presentados en la Sección 4 demuestran que Merge Sort optimizado y Quick Sort exhiben tiempos de ejecución comparables en el caso promedio, la elección recae en **Quick Sort** por razones de uso de memoria. Merge Sort requiere un arreglo auxiliar del mismo tamaño que la entrada para realizar la fase de combinación, incurriendo en una complejidad espacial de $O(n)$. Quick Sort, en cambio, opera directamente sobre el arreglo original mediante intercambios in-place, con una complejidad espacial de $O(1)$ en cuanto a memoria adicional (sin considerar la pila de recursión). En un sistema donde el tamaño del conjunto de deportistas puede crecer arbitrariamente, esta diferencia resulta determinante para mantener un uso eficiente de la memoria disponible.

6.2.2. Algoritmo de búsqueda: búsqueda por interpolación y búsqueda por rangos

Para la búsqueda de un deportista por identificador se adopta la **búsqueda por interpolación** como algoritmo predeterminado, ya que, en el caso de este proyecto, la búsqueda de un elemento exacto se hace con el `id`, y este criterio al ser un tipo de dato que se puede distribuir numéricamente y uniforme a lo largo del arreglo, es donde mejor se comporta este algoritmo a diferencia de los demás.

Para la consulta por rango de puntaje, la **búsqueda binaria por rangos** es la única alternativa implementada capaz de retornar directamente los límites inferior y superior del intervalo en el arreglo ordenado, lo que la convierte en la elección natural para esta funcionalidad.

7. Conclusiones

A partir del análisis teórico y experimental realizado, se extraen las siguientes conclusiones respecto al comportamiento de los algoritmos estudiados.

Superioridad asintótica de Divide y Vencerás en ordenamiento

Los resultados experimentales confirmaron de forma contundente la ventaja de los algoritmos $O(n \log n)$ frente a los algoritmos $O(n^2)$ del Proyecto 1. A medida que el tamaño de entrada crece, la brecha entre ambas familias se vuelve cada vez más pronunciada. Este comportamiento es consistente con la superioridad asintótica predicha por el análisis teórico y refuerza la importancia de seleccionar algoritmos de menor complejidad para conjuntos de datos de gran tamaño.

Excepción en el mejor caso: la ventaja de los iterativos optimizados

El escenario de datos ya ordenados reveló un resultado llamativo: los algoritmos iterativos con detección temprana de terminación superan en velocidad a los algoritmos de Divide y Vencerás, alcanzando su mejor caso de $O(n)$ mediante un único recorrido lineal sin intercambios. Este resultado ilustra que la complejidad asintótica no es el único criterio relevante en la selección de algoritmos: el estado inicial de los datos puede invertir la jerarquía de rendimiento entre familias de distinto orden de complejidad.

Impacto de las optimizaciones dentro de una misma clase de complejidad

Dentro de los algoritmos de Divide y Vencerás, los experimentos evidenciaron que las optimizaciones tienen un efecto práctico significativo sin alterar el orden asintótico. En Merge Sort, la incorporación de un umbral para delegar a Insertion Sort los subarreglos pequeños redujo los tiempos de ejecución de forma consistente. En Quick Sort, la elección del pivote demostró ser determinante: la estrategia de mediana de tres ofreció el mejor equilibrio entre rendimiento y estabilidad, mientras que los pivotes de primer y último elemento se degradaron a $O(n^2)$ frente a arreglos ya ordenados o en orden inverso.

Comportamiento de los algoritmos de búsqueda

Los algoritmos de búsqueda basados en Divide y Vencerás demostraron ser significativamente más eficientes que la búsqueda secuencial del Proyecto 1. La búsqueda binaria recursiva e iterativa, la búsqueda exponencial y la búsqueda binaria para rangos exhibieron un comportamiento $O(\log n)$ consistente con sus análisis teóricos. La búsqueda por interpolación mostró ventajas sobre la búsqueda binaria en distribuciones uniformes de datos, estimando con mayor precisión la posición del elemento buscado; sin embargo, su rendimiento se degrada frente a distribuciones sesgadas, según la teoría.

Quick Select como alternativa eficiente a la selección por ordenamiento

Quick Select demostró ser una alternativa eficiente para obtener el k -ésimo elemento, operando en tiempo promedio $O(n)$ frente al $O(n \log n)$ del ordenamiento previo. Su peor caso de $O(n^2)$, análogo al de Quick Sort con mala elección de pivote, puede mitigarse mediante estrategias de selección de pivote adecuadas, tal como se analizó en la sección correspondiente.

Validación de la teoría en la práctica

El análisis experimental corroboró, en términos generales, las predicciones del análisis teórico de complejidad para los algoritmos estudiados. Las curvas de tiempo de ejecución se ajustaron a los órdenes asintóticos esperados, confirmando que el análisis de complejidad constituye una herramienta confiable para anticipar el comportamiento global de los algoritmos.

La principal salvedad corresponde al peor caso de la búsqueda por interpolación. Aunque teóricamente este algoritmo puede degradarse hasta un comportamiento lineal, dicho escenario depende de distribuciones desfavorables de los criterios y no se reproduce necesariamente mediante la

simple búsqueda de un identificador inexistente. En este trabajo, con el fin de mantener una comparación homogénea entre los algoritmos de búsqueda exacta, se evaluó dicho caso utilizando `id` como criterio de búsqueda, por lo que el peor caso experimental de interpolación debe interpretarse como una aproximación uniforme dentro del diseño experimental, más que como una reproducción estricta de su peor caso teórico.

Los resultados obtenidos en ambos proyectos refuerzan, además, que la elección del algoritmo más adecuado no depende únicamente de su complejidad teórica, sino también de las características particulares de los datos de entrada y del contexto de aplicación.

8. Contribución por integrante

A continuación se detalla la contribución realizada por cada uno de los integrantes del grupo en el desarrollo del proyecto.

Franco Aguilar

El estudiante **Franco Aguilar** contribuyó al proyecto con las siguientes tareas:

Migración del sistema de menús a `getopt()`

Reemplazo del sistema de menús interactivo por un sistema basado en la función `getopt()` de C, permitiendo el uso de argumentos de línea de comandos.

Implementación de algoritmos de búsqueda

Implementación de los algoritmos de búsqueda.

- Creación de rama `search_algorithms`
- Búsqueda binaria recursiva estándar
- Búsqueda binaria para rangos
- Búsqueda exponencial
- Búsqueda por interpolación
- Merge a rama principal

Análisis del comportamiento de algoritmos de búsqueda

Se deberá analizar experimentalmente el comportamiento de los algoritmos de búsqueda implementados en el peor caso y caso promedio.

Implementación de nuevas opciones a `getopt()`

- Agregar nuevas opciones solicitadas en la tarea.
- Adaptar las opciones para que funcionen con algoritmos predeterminados

Deportistas por rango

Mostrar los deportistas cuyo puntaje se encuentre dentro de un rango determinado.

Milton Hernández

El estudiante **Milton Hernández** contribuyó al proyecto con las siguientes tareas:

Adaptación del repositorio del Proyecto 1

Creación de fork del repositorio del Proyecto 1 y adaptación al nuevo contexto de trabajo con los nuevos integrantes del grupo.

Nota: Se mantiene el sistema de deportistas desarrollado previamente como base para integrar los nuevos algoritmos.

Estructura inicial del informe LaTeX

Preparación del documento LaTeX con la estructura base del informe según los requisitos de la tarea.

Implementación de Quick Sort con 4 variantes

Implementación de Quick Sort utilizando el esquema de partición de Lomuto con cuatro estrategias de selección de pivote.

- Creación de rama `quick_sort`
- Implementación del esquema de partición de Lomuto
- Variantes de pivote (Primer elemento, Último elemento, Mediana de tres, Elemento aleatorio)
- Análisis de comportamiento en mejor, peor y caso promedio y elección de pivote para implementación final
- Merge a rama principal

Implementación de Quick Select

Implementación del algoritmo Quick Select para encontrar el k-ésimo elemento sin ordenar todo el arreglo.

- Creación de rama `quick_select`
- Reutilización del esquema de partición de Quick Sort
- Integración con funcionalidad “k-ésimo mejor deportista”
- Merge a rama principal

Análisis de complejidad temporal de algoritmos iterativos de ordenamiento

Análisis experimental del comportamiento de `quicksort` y `mergesort` en el mejor, peor y caso promedio.

Comparación con los de los algoritmos implementados en la tarea anterior.

Ranking de deportistas

Generar un ranking con los mejores N deportistas, utilizando el puntaje como criterio.

Luis Muñoz

El estudiante **Luis Muñoz** contribuyó al proyecto con las siguientes tareas:

Estructura inicial de la presentación

Creación del esqueleto de la presentación.

- Definición de estructura y flujo de contenido
- Creación de plantillas de diapositivas

Advertencia: La presentación no puede superar los 10 minutos.

Implementación de Merge Sort

Implementación de Merge Sort en sus dos variantes: clásica y optimizada con Insertion Sort para subarreglos pequeños.

- Creación de rama `merge_sort`
- Implementación de la versión clásica con división recursiva
- Función `merge()` para combinar subarreglos ordenados
- Versión optimizada con umbral para Insertion Sort
- Pruebas experimentales con distintos umbrales
- Merge a rama principal tras elegir el mejor umbral experimentalmente

Análisis del comportamiento de quickselect

Se deberá analizar el comportamiento en el mejor y peor caso.

9. Anexos

Código 1. Elección del pivote para QuickSort

```

1  /* Eleccion del pivote */
2  int pivot_pos = 0;
3  switch (pivot_type) {
4      case 1:
5          pivot_pos = left;
6          break;
7      case 2:
8          pivot_pos = right;
9          break;
10     case 3:
11         pivot_pos = median_of_three(V, left, right, comp_f);
12         break;
13     case 4:
14         pivot_pos = rand() % (right - left + 1) + left;
15         break;
16 }
17
18 /* Mediana de 3 */
19 int median_of_three(Player V[], int left, int right, int (*comp_f)(Player *, Player *))
20 {
21     int mid = (left + right) / 2;
22     if (comp_f(&V[left], &V[mid]) > 0) { // a > b
23         if (comp_f(&V[mid], &V[right]) > 0) { // a > b > c, mediana es b
24             return mid;
25         } else if (comp_f(&V[left], &V[right]) > 0) { // a > c >= b, mediana es c
26             return right;
27         } else { // c >= a > b, mediana es a
28             return left;
29         }
30     } else { // a <= b
31         if (comp_f(&V[left], &V[right]) > 0) { // b >= a > c, mediana es a
32             return left;
33         } else if (comp_f(&V[mid], &V[right]) > 0) { // b > c >= a, mediana es c
34             return right;
35         } else { // c >= b >= a, mediana es b
36             return mid;
37         }
38     }
39 }

```

9.1. Experimentos con Quicksort

Cuadro 1. Tiempos de ejecución (en segundos) de las cuatro variantes de pivote de Quicksort sobre un arreglo desordenado de hasta 1 000 000 elementos.

N	Primer elemento	Último elemento	Mediana de 3	Aleatorio
1	0.000000	0.000000	0.000000	0.000000
41 667	0.003330	0.003400	0.003408	0.003814
83 334	0.007141	0.007083	0.007067	0.008053
125 000	0.011038	0.011388	0.011149	0.012483
166 667	0.015325	0.015590	0.015224	0.017014
208 334	0.019587	0.020327	0.019645	0.021669
250 000	0.024003	0.024472	0.023751	0.026400
291 667	0.028150	0.027936	0.027959	0.031200
333 334	0.032451	0.032914	0.032373	0.036073
375 000	0.037226	0.036969	0.036823	0.040977
416 667	0.042333	0.041714	0.041766	0.045977
458 333	0.046449	0.046490	0.045851	0.051010
500 000	0.052213	0.051138	0.050894	0.056159
541 667	0.055555	0.055321	0.055124	0.061336
583 333	0.060037	0.060520	0.059542	0.066524
625 000	0.064624	0.066366	0.064633	0.071724
666 667	0.069890	0.070719	0.069883	0.077012
708 333	0.074531	0.074916	0.073959	0.082404
750 000	0.078816	0.080144	0.078841	0.087680
791 666	0.085814	0.084799	0.083484	0.093176
833 333	0.089257	0.093192	0.089447	0.098454
875 000	0.093786	0.093912	0.092850	0.103771
916 666	0.098090	0.099177	0.097574	0.109352
958 333	0.104806	0.102728	0.103007	0.114742
1 000 000	0.111140	0.109411	0.107273	0.120314

Cuadro 2. Tiempos de ejecución (en segundos) de las cuatro variantes de pivote de Quicksort sobre un arreglo previamente ordenado de hasta 50 000 elementos.

N	Primer elemento	Último elemento	Mediana de 3	Aleatorio
2 084	0.002821	0.005636	0.000053	0.000096
4 167	0.011216	0.022800	0.000111	0.000198
6 250	0.025136	0.051265	0.000183	0.000306
8 334	0.044641	0.091310	0.000238	0.000418
10 417	0.069793	0.142839	0.000311	0.000533
12 500	0.100629	0.205307	0.000391	0.000648
14 584	0.136598	0.279809	0.000450	0.000766
16 667	0.178826	0.366512	0.000507	0.000882
18 750	0.226386	0.463641	0.000584	0.001001
20 833	0.280109	0.572521	0.000663	0.001122
22 917	0.339712	0.693150	0.000743	0.001243
25 000	0.403578	0.826304	0.000834	0.001363
27 083	0.473774	0.971292	0.000879	0.001486
29 167	0.552309	1.128824	0.000958	0.001613
31 250	0.636738	1.300447	0.001006	0.001735
33 333	0.726952	1.485349	0.001150	0.001921
35 416	0.823885	1.679442	0.001229	0.002051
37 500	0.924564	1.884134	0.001323	0.002183
39 583	1.035054	2.104206	0.001412	0.002321
41 666	1.148560	2.336949	0.001494	0.002444
43 750	1.269816	2.577580	0.001590	0.002579
45 833	1.397713	2.832901	0.001676	0.002707
47 916	1.531550	3.100269	0.001793	0.002840
50 000	1.669994	3.384895	0.001859	0.002971

Cuadro 3. Tiempos de ejecución (en segundos) de las cuatro variantes de pivote de Quicksort sobre un arreglo en orden inverso de hasta 50 000 elementos.

N	Primer elemento	Último elemento	Mediana de 3	Aleatorio
2 084	0.005403	0.005394	0.000114	0.000111
4 167	0.021538	0.021497	0.000255	0.000233
6 250	0.048321	0.048370	0.000406	0.000364
8 334	0.086111	0.086156	0.000562	0.000496
10 417	0.134148	0.134187	0.000727	0.000632
12 500	0.193196	0.193387	0.000895	0.000768
14 584	0.263260	0.263252	0.001066	0.000911
16 667	0.343293	0.343365	0.001243	0.001049
18 750	0.434649	0.434730	0.001410	0.001192
20 833	0.537103	0.536447	0.001594	0.001331
22 917	0.648835	0.648374	0.001771	0.001479
25 000	0.773403	0.772198	0.001953	0.001622
27 083	0.910271	0.906600	0.002139	0.001774
29 167	1.059122	1.053086	0.002321	0.001920
31 250	1.220292	1.215089	0.002512	0.002076
33 333	1.394196	1.386571	0.002709	0.002214
35 416	1.573796	1.569751	0.002892	0.002370
37 500	1.770812	1.766749	0.003073	0.002522
39 583	1.979956	1.976280	0.003270	0.002670
41 666	2.198634	2.197493	0.003473	0.002822
43 750	2.430521	2.430134	0.003666	0.002972
45 833	2.675791	2.680852	0.003870	0.003139
47 916	2.937548	2.932643	0.004057	0.003280
50 000	3.200714	3.202993	0.004255	0.003433

9.2. Teorema Maestro

El **Teorema Maestro** constituye una herramienta fundamental para determinar la complejidad temporal de algoritmos que siguen el paradigma de Divide y Vencerás, ya que proporciona un mecanismo sistemático para resolver ecuaciones de recurrencia de la forma:

$$T(n) = \begin{cases} c & \text{si } n \leq n_0 \\ aT(n/b) + f(n) & \text{si } n > n_0 \end{cases}$$

donde $a \geq 1$, $b > 1$ y $f(n)$ es una función asintóticamente positiva. Los parámetros a y b representan, respectivamente, el número de subproblemas en que se divide el problema original y el factor por el cual se reduce el tamaño de cada subproblema; mientras que $f(n)$ captura el costo adicional del trabajo realizado fuera de las llamadas recursivas (por ejemplo, la división del problema y la combinación de resultados). Bajo estas condiciones, el teorema establece los tres casos siguientes:

1. Si $f(n) = O(n^{\log_b a - \epsilon})$ para alguna constante $\epsilon > 0$, el trabajo de combinación es asintóticamente menor que el costo de los subproblemas y, por tanto:

$$T(n) = O(n^{\log_b a})$$

2. Si $f(n) = O(n^{\log_b a} \log^k n)$ para alguna constante $k \geq 0$, ambas contribuciones son del mismo orden y:

$$T(n) = O(n^{\log_b a} \log^{k+1} n)$$

3. Si $f(n) = \Omega(n^{\log_b a + \epsilon})$ para alguna constante $\epsilon > 0$, y se cumple la condición de regularidad $af(n/b) \leq cf(n)$ para alguna constante $c < 1$ y todo n suficientemente grande, el costo de combinación domina y:

$$T(n) = O(f(n))$$

Para los algoritmos analizados en este informe resulta conveniente emplear la **versión simplificada** del teorema, aplicable cuando $f(n)$ es una función polinómica de la forma cn^k :

$$T(n) = \begin{cases} c n^k & \text{si } 1 \leq n < b \\ a T\left(\frac{n}{b}\right) + c n^k & \text{si } n \geq b \end{cases} \implies T(n) \in \begin{cases} O(n^k) & \text{si } \log_b a < k \\ O(n^k \log n) & \text{si } \log_b a = k \\ O(n^{\log_b a}) & \text{si } \log_b a > k \end{cases}$$

En esencia, el resultado depende de la relación entre $\log_b a$ (exponente que refleja cuánto crece el árbol de recursión) y k (exponente que caracteriza el costo por nivel): si el árbol de recursión domina, la complejidad queda determinada por su altura; si el costo por nivel domina, la complejidad viene dada por dicho costo; y si ambos están equilibrados, aparece un factor logarítmico adicional.

■ Referencias

- [1] M. D. McIlroy, «A Killer Adversary for Quicksort», *Software: Practice and Experience*, vol. 29, n.º 4, págs. 341-344, 1999.